

src/incidence.cpp — Function-by-Function Explanation

A-Phi Solver project — code reference

September 2026

Contents

Overview	1
SparseMatrix::coalesce	1
SparseMatrix::to_dense	2
multiply	2
is_zero_matrix	2
build_gradient_matrix	3
build_curl_matrix	3

Overview

This file implements the two discrete topological operators that Phase 02 of docs/ROADMAP.md calls for: the discrete **gradient/incidence matrix G** (node values $\rightarrow$ edge values) and the discrete **curl matrix C** (edge values $\rightarrow$ face values), plus a small amount of generic sparse-matrix machinery (SparseMatrix’s member functions, multiply, is_zero_matrix) needed to build and check them. These matrices are what the classical $C * G = 0$ identity — the discrete statement that “the curl of a gradient is zero” — is checked against, which is Phase 02’s own numerical exit criterion.

The SparseMatrix type itself (a plain COO/triplet list of (row, col, value) entries) is declared in include/aphi_solver/incidence.hpp; its add member is a trivial one-liner defined inline in the header, so it does not appear in this file. The six functions that *are* implemented in incidence.cpp are covered below in the order they appear in the file.

All six are exercised by tests/test_incidence.cpp (15/15 checks passing) and tests/test_gmsh_reader.cpp (7/7 checks passing, on a mesh read from an actual Gmsh file rather than built programmatically).

SparseMatrix::coalesce

```
void SparseMatrix::coalesce()
```

A member function of SparseMatrix that merges duplicate (row, col) entries by summing their values. It builds a `std::map<std::pair<int,int>, double>` keyed by (row, col), accumulates every existing entry’s value into the matching map slot, then replaces entries with one entry per distinct (row, col) pair (using the map’s natural sort order, so the result is also sorted by row then column — convenient for inspection, not required for correctness).

Why it's needed: `build_gradient_matrix` and `build_curl_matrix` below never produce duplicate entries by construction (each row of $\mathbf{G}$ or $\mathbf{C}$ touches each column at most once), but a matrix *product* — see `multiply` below — generally does produce several contributions landing on the same (`row`, `col`) position, which need to be summed rather than kept as separate list entries. `multiply` calls `coalesce()` on its result before returning it.

SparseMatrix::to_dense

```
std::vector<std::vector<double>> SparseMatrix::to_dense() const
```

Converts the sparse triplet list into an ordinary dense 2-D `std::vector<std::vector<double>>` of size `rows` x `cols`, initialized to zero and then filled in (with `+=`, so any un-coalesced duplicate entries are still summed correctly) from `entries`.

This exists purely for small-scale debugging and test convenience — the header comment is explicit that it should never be used on a production-size mesh, since a dense array grows as `rows * cols` while the matrices themselves are sparse (each row of $\mathbf{G}$ has only 2 nonzeros, each row of $\mathbf{C}$ only 3). It is not called anywhere in `incidence.cpp` itself; it is a convenience for callers, such as tests, that want to eyeball a small matrix directly.

multiply

```
SparseMatrix multiply(const SparseMatrix& a, const SparseMatrix& b)
```

Computes the ordinary matrix product $\mathbf{a} * \mathbf{b}$ for two `SparseMatrix` values, returning a new `SparseMatrix` of size `a.rows` x `b.cols`. This is what $\mathbf{C} * \mathbf{G}$ is computed with, to check the $\mathbf{CG} = \mathbf{0}$ identity.

How it works: rather than a naive triple-nested loop over dense indices, it first buckets $\mathbf{b}$'s entries by row into a `std::map<int, std::vector<SparseMatrix::Entry>>` (`b_by_row`) — so “give me all of $\mathbf{b}$'s entries in row k ” is a fast lookup. Then, for every entry (`ae.row`, `ae.col`, `ae.value`) of $\mathbf{a}$, it looks up `b_by_row[ae.col]` (the row of $\mathbf{b}$ that lines up with this entry's column, as matrix multiplication requires) and adds `ae.value * be.value` into the result at (`ae.row`, `be.col`) for every matching entry `be`. Finally it calls `result.coalesce()` so that any position touched by more than one contribution ends up as a single summed entry.

The header comment is explicit that this is not an optimized sparse-matrix product — it is correctness-focused, sized for the small test meshes Phase 02 targets, with a real sparse-linear-algebra library expected to take over once Phase 04/05 need to assemble and solve production-size systems.

is_zero_matrix

```
bool is_zero_matrix(const SparseMatrix& m, double tol)
```

Returns `true` if and only if every entry in `m.entries` has magnitude no larger than `tol` (default `1e-9`, set in the header). It simply loops over every entry and returns `false` at the first one exceeding the tolerance; if the loop finishes without finding one, it returns `true`.

This is the function that turns the $\mathbf{C} * \mathbf{G} = \mathbf{0}$ identity into an actual pass/fail check: after computing `multiply(C, G)`, the test suite calls `is_zero_matrix` on the result. A nonzero tolerance is necessary rather than an exact `== 0.0` comparison because, even though this project's specific construction of $\mathbf{C}$ and $\mathbf{G}$ makes the entries of $\mathbf{C} * \mathbf{G}$ cancel exactly in exact arithmetic (see `build_curl_matrix` below), floating-point summation order inside `multiply`'s `coalesce` step could in principle leave a tiny nonzero residual — `1e-9` comfortably separates real cancellation-to-zero from any such residual while still catching a genuinely wrong $\mathbf{C}$ or $\mathbf{G}$ construction, which would produce residuals many orders of magnitude larger.

build_gradient_matrix

```
SparseMatrix build_gradient_matrix(const Mesh& mesh)
```

Builds the discrete gradient operator $\mathbf{G}$, mapping node values to edge values, with shape `num_edges x num_nodes`. For each global edge e , the mesh’s canonical orientation convention (edges are always stored as a pair (i, j) with $i < j$, established in `mesh.hpp/mesh.cpp`) gives a fixed “positive direction” $i \rightarrow j$, and this function sets row e of $\mathbf{G}$ to $G(e, i) = -1$ and $G(e, j) = +1$. Every other entry in that row is implicitly zero (the `SparseMatrix` never stores them).

Why this is “the gradient”: for a scalar function ϕ defined at mesh nodes, $(\mathbf{G} * \phi)(e) = \phi(j) - \phi(i)$ — exactly the finite-difference approximation to the directional derivative of ϕ along edge e , in its positive direction. This is the standard node-edge incidence matrix used throughout the tree-cotree gauging literature already cited in this project (Munteanu’s papers), and it is what Phase 03’s spanning-tree machinery will build on directly.

build_curl_matrix

```
SparseMatrix build_curl_matrix(const Mesh& mesh)
```

Builds the discrete curl operator $\mathbf{C}$, mapping edge values to face values, with shape `num_faces x num_edges`. For each global (triangular) face f , the mesh’s canonical face orientation (faces are always stored as a triple (a, b, c) with $a < b < c$) defines a fixed positive traversal $a \rightarrow b \rightarrow c \rightarrow a$. The function looks up the global edge index for each of the face’s three sides via `mesh.find_edge` and sets:

```
C(f, edge(a,b)) = +1  
C(f, edge(b,c)) = +1  
C(f, edge(a,c)) = -1
```

The last sign is -1 , not $+1$, because the edge (a, c) is stored canonically as going from the smaller index to the larger one — the same direction as $a \rightarrow c$ — but the face’s positive traversal actually needs to go the other way around that side, $c \rightarrow a$, to close the triangle $a \rightarrow b \rightarrow c \rightarrow a$; the minus sign accounts for that reversal so the whole row correctly represents the signed sum of the face’s boundary edges.

Why this is “the curl”: this is the standard discrete Stokes / boundary operator for a triangulated surface — summing a vector field’s values around a closed loop (here, a face’s boundary) is the discrete analogue of the closed line integral of $\mathbf{A}$ around that loop, which by Stokes’ theorem equals the flux of $\text{curl}(\mathbf{A})$ through that face. This construction is sourced to Bossavit (1998) Ch. 5 (already cited in `docs/REFERENCES.md`) and is not specific to any one implementation.

Why $\mathbf{C} * \mathbf{G} = \mathbf{0}$ holds exactly: for any two edges of the same face that share a vertex, that shared vertex’s node value appears in both edges’ contributions to $(\mathbf{G} * \phi)$, with opposite signs once $\mathbf{C}$ ’s $+1/+1/-1$ pattern is applied around the closed loop — the three $\phi(\text{vertex})$ terms around the triangle telescope and cancel completely. This is exactly the discrete statement that $\text{curl}(\text{grad}(\phi)) = 0$, and it is what `tests/test_incidence.cpp` and `tests/test_gmsh_reader.cpp` verify numerically (via `multiply` and `is_zero_matrix` above) as Phase 02’s exit criterion.